4 · Exp 4 — Delete from an array (beginning, any position, end)

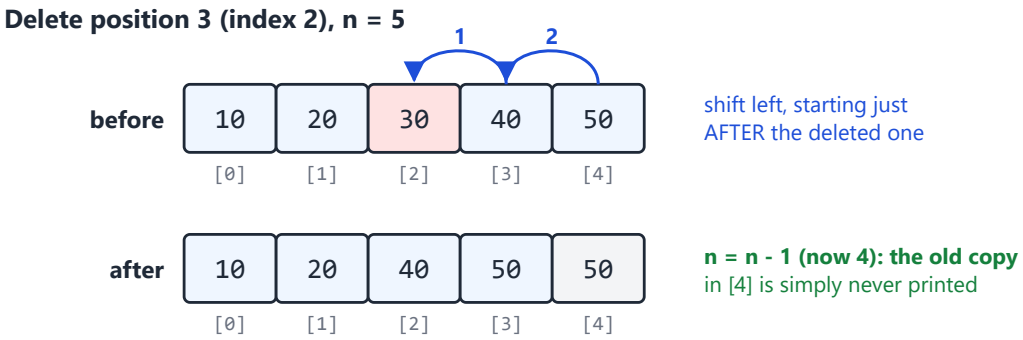
The mirror image of Exp 3: close the gap by shifting **left**, then n = n − 1.

Pick the element → **Shift the rest left** → n = n − 1 → print 0..n−1

4.1 The idea

GUESS FIRST

In Exp 3 the shift loop had to run from the **end** backwards. For deletion (shift left), which end must the loop start from, and why?



Delete position 3: shift from just after the gap, moving forwards.

When shifting **left**, start right after the deleted element and move **forward**: `arr[i] = arr[i+1]` for i = pos−1 … n−2. Going the other way would copy 50 over 40 before 40 has moved. **Rule of thumb: always copy the element that is about to be overwritten first.**

The last element is never erased; after `n--` the program simply never prints index n. That's why delete-at-end is just `n--`: O(1), no shifting.

4.2 The manual's bug

The PROGRAM listing (manual page 9) asks **"Enter value:"** in all three branches and reads a number it never uses. Deleting needs only a *position*. With the manual's own Sample Input 1 (`5`, `10 20 30 40 50`, `1`) that listing stops at "Enter value:" and waits for input the sample never gives. The IMPLEMENTATION screenshot (page 11) has no such line. This pack uses the screenshot's logic, plus a position check, and prints which element was deleted.

4.3 The program (corrected)

E04_ARRAY_DELETE.CPP

```
#include <iostream>
using namespace std;

int main()
```

```
{
    int arr[100], n, pos, choice;           // FIX: no "value" - deletion needs no value

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter array elements:\n";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    if (n <= 0)                             // FIX: nothing to delete
    {
        cout << "Array is empty. Deletion not possible.";
        return 0;
    }

    cout << "\n1. Delete at Beginning";
    cout << "\n2. Delete at Any Position";
    cout << "\n3. Delete at End";
    cout << "\nEnter your choice: ";
    cin >> choice;

    if (choice == 1)
    {
        cout << "Deleted element: " << arr[0];
        for (int i = 0; i < n - 1; i++)     // shift left, first element first
            arr[i] = arr[i + 1];
        n--;
    }
    else if (choice == 2)
    {
        cout << "Enter Position: ";
        cin >> pos;

        if (pos < 1 || pos > n)             // FIX: positions 1..n are valid
        {
            cout << "Invalid position.";
            return 0;
        }

        cout << "Deleted element: " << arr[pos - 1];
        for (int i = pos - 1; i < n - 1; i++)
            arr[i] = arr[i + 1];
        n--;
    }
    else if (choice == 3)
    {
        cout << "Deleted element: " << arr[n - 1];
        n--;                                // just forget the last one
    }
    else
    {
        cout << "Invalid choice.";
        return 0;
    }

    cout << "\nUpdated Array:\n";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
```

```
    return 0;
}
```

The manual's three sample inputs:

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Delete at Beginning
2. Delete at Any Position
3. Delete at End
Enter your choice: 1
Deleted element: 10
Updated Array:
20 30 40 50
```

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Delete at Beginning
2. Delete at Any Position
3. Delete at End
Enter your choice: 2
Enter Position: 3
Deleted element: 30
Updated Array:
10 20 40 50
```

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Delete at Beginning
2. Delete at Any Position
3. Delete at End
Enter your choice: 3
Deleted element: 50
Updated Array:
10 20 30 40
```

A bad position is refused:

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Delete at Beginning
2. Delete at Any Position
3. Delete at End
Enter your choice: 2
```

```
Enter Position: 6
Invalid position.
```

Case	Loop	Then
Beginning	`for (i = 0; i < n-1; i++) arr[i] = arr[i+1];`	`n--`
Position	`for (i = pos-1; i < n-1; i++) arr[i] = arr[i+1];`	`n--`
End	(none)	`n--`

Valid deletion positions are **1 to n** (not n+1: there's nothing there to delete). With n = 0 there's nothing to delete at all: **underflow**.

> **WRITE IN YOUR RECORD**
>
> **Aim:** To delete an element from an array at the beginning, at a given position, and at the end.
>
> **Algorithm:** read n and the elements → read choice → *beginning:* shift elements 1…n−1 one place left → *position:* check 1 ≤ pos ≤ n, shift elements after pos−1 one place left → *end:* nothing to shift → `n--` → print.
>
> **Complexity:** beginning O(n), position O(n) (worst case), end O(1). Space O(1).
>
> **Result:** 10 20 30 40 50 became 20 30 40 50 (beginning), 10 20 40 50 (position 3, deleted 30), 10 20 30 40 (end).

4.4 Viva questions

Question	Answer
Why does deletion at the end need no shifting?	Reducing n hides the last element; no gap is created.
Is the deleted value really erased from memory?	No. It stays in the slot until overwritten; the program just stops counting it.
What is underflow here?	Trying to delete from an empty array (n = 0).
Insertion shifts from the end, deletion from the front. Why?	So no element is overwritten before it has been copied.
Time complexity of deleting a value you first have to search for?	O(n): the search is O(n) and the shift is O(n), so O(n) overall.

4.5 If the examiner changes the question

Beginning, position and end are the experiment itself. The other two items:

Delete a given value (search, then shift):

```c
// Search for value; if found, delete it (first occurrence) and shift left
int deleteByValue(int arr[], int n, int value)
{
    int pos = -1;
    for (int i = 0; i < n; i++)          // linear search
```

```
        if (arr[i] == value)
        {
            pos = i;
            break;
        }

    if (pos == -1)
    {
        cout << value << " not found in array" << endl;
        return n;
    }
    for (int i = pos; i < n - 1; i++)
        arr[i] = arr[i + 1];
    return n - 1;
}
```

Menu-driven insertion and deletion on one array. A full program combining Exp 3 and Exp 4, with a repeating menu (the `do … while` pattern from Exp 10):

E04B_ARRAY_MENU.CPP

```
// Exp 4 assignment: menu-driven insertion AND deletion on one array
#include <iostream>
using namespace std;

int arr[100], n = 0;

void display()
{
    if (n == 0)
        cout << "Array is empty." << endl;
    else
    {
        cout << "Array: ";
        for (int i = 0; i < n; i++)
            cout << arr[i] << " ";
        cout << endl;
    }
}

void insertAt(int pos, int value)            // pos is 1-based
{
    if (n >= 100)
        cout << "Array is full." << endl;
    else if (pos < 1 || pos > n + 1)
        cout << "Invalid position." << endl;
    else
    {
        for (int i = n; i >= pos; i--)
            arr[i] = arr[i - 1];
        arr[pos - 1] = value;
        n++;
    }
}

void deleteAt(int pos)
{
    if (n == 0)
        cout << "Array is empty." << endl;
    else if (pos < 1 || pos > n)
```

```
        cout << "Invalid position." << endl;
    else
    {
        cout << "Deleted element: " << arr[pos - 1] << endl;
        for (int i = pos - 1; i < n - 1; i++)
            arr[i] = arr[i + 1];
        n--;
    }
}

int main()
{
    int choice, pos, value;
    do
    {
        cout << "\n1. Insert  2. Delete  3. Display  4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter position and value: ";
            cin >> pos >> value;
            insertAt(pos, value);
        }
        else if (choice == 2)
        {
            cout << "Enter position: ";
            cin >> pos;
            deleteAt(pos);
        }
        else if (choice == 3)
            display();
        else if (choice != 4)
            cout << "Invalid choice!" << endl;
    } while (choice != 4);

    return 0;
}
```

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Insert  2. Delete  3. Display  4. Exit
Enter your choice: 1
Enter position and value: 1 10
Enter your choice: 1
Enter position and value: 2 30
Enter your choice: 1
Enter position and value: 2 20
Enter your choice: 3
Array: 10 20 30
Enter your choice: 2
Enter position: 1
Deleted element: 10
Enter your choice: 3
Array: 20 30
Enter your choice: 2
Enter position: 5
Invalid position.
Enter your choice: 4
```

CHECK 4.1

`arr` = `7 3 9 1`, n = 4. Delete position 2. Write the assignments the loop performs, the final n, and the printed array.

CHECK 4.2

Someone writes the position loop as `for (i = pos-1; i < n; i++) arr[i] = arr[i+1];`. What goes wrong?

Answers

Check 4.1: i = 1: `arr[1] = arr[2]` (9); i = 2: `arr[2] = arr[3]` (1); stop at i = 3. n = 3 → **7 9 1**.

Check 4.2: On the last pass (i = n−1) it reads `arr[n]`, one past the data: junk (or past the end of the array if n = 100). The result still prints correctly, because n−1 elements are printed, but it reads memory it shouldn't. The correct bound is `i < n-1`.